

# Contents

|                  |          |
|------------------|----------|
| <b>1 图论</b>      | <b>1</b> |
| 1.1 一些概念         | 1        |
| 1.2 树的重心         | 2        |
| 1.3 树的直径         | 4        |
| 1.4 最近公共祖先 lca   | 6        |
| 1.4.1 重链剖分       | 6        |
| 1.4.2 树链剖分 (线段树) | 9        |
| 1.4.3 欧拉序 +RMQ   | 11       |
| 1.4.4 倍增求 LCA    | 12       |
| 1.4.5 树上差分       | 13       |
| 1.5 树上启发式合并      | 14       |
| 1.6 虚树           | 16       |
| 1.7 树哈希          | 17       |
| 1.8 网络流          | 19       |
| 1.8.1 拆点         | 19       |
| 1.8.2 最大流 Dinic  | 19       |
| 1.8.3 最小费用最大流    | 22       |

## 1 图论

### 1.1 一些概念

基图：即把有向图中的有向边改为无向边得到的图称作基图

弱连通：对于有向图中两点  $(u, v)$ ，如果我们转化为基图后  $(u, v)$  联通，那么称  $(u, v)$  弱联通

完全图：任意两点间都有一条无向边的图称作完全图， $n$  个点的完全图记作  $K_n$

子图：对于一个图  $G = (V, E)$ ，如果另一个图  $G' = (V', E')$  满足  $V' \subseteq V$  且  $E' \subseteq E$  则  $G'$  称作  $G$  的子图

导出子图：选定一个点集  $V$ ，再加上两端都在  $V$  中的边，即为点集  $V$  的导出子图

平面图：将所有点和边铺在平面上，可以满足每条边之间两两不相交的图称作平面图，对于平面图  $G = (V, E)$  满足  $E \leq 3V - 6$  且根据欧拉公式，平面图满足  $V - E + F = C + 1$  其中  $F$  是面的个数， $C$  是图连通块的个数

## 1.2 树的重心

我们不妨设树的大小为  $n$

树的重心定义为，当树以重心为根时，每个子树的大小不超出  $n/2$ 。

即最大的子树大小  $siz * 2 \leq n$

性质：一棵树最多只能有两个重心。

求重心的方法：

在我们求出以任意一点为根的子树大小后，我们从根开始 *dfs*。如果某个点的所有儿子的子树大小都  $\leq n/2$ ，那么该点为重心，直接返回。否则我们递归子树大小超过  $n/2$  的儿子即可（显然只有一个）。

```
struct TreeCentroid
{
    int n;
    std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
    std::vector<int> siz; // 子树大小

    TreeCentroid(int n) : n(n), g(n + 1), siz(n + 1) {}

    void addEdge(int u, int v, int w)
    {
        g[u].emplace_back(v, w);
        g[v].emplace_back(u, w);
    }

    void dfsSize(int u, int fa)
    {
        siz[u] = 1;
        for (const auto &[v, w] : g[u])
        {
            if (v != fa)
            {
                dfsSize(v, u);
                siz[u] += siz[v];
            }
        }
    }

    int findCentroid(int u)
    {
        dfsSize(u, 0);
        return getRoot(u, 0, siz[u]);
    }

private:
    int getRoot(int u, int fa, int total_size)
    {
        for (const auto &[v, w] : g[u])
        {
            if (v != fa && 2 * siz[v] > total_size)
            {
                return getRoot(v, u, total_size);
            }
        }
        return u;
    }
}
```

};

### 1.3 树的直径

注意初始化时传入的图因该是 1-base

可直接获取直径长度，其中一条直径上的所有点，以及每个点到父节点直接连边的权值

两次 bfs 会更改 fa[] 数组，其中 st 表示返回路径的起点，en 表示终点，d 是直径长度

```
struct TreeDiameter{
    int n,d = -1,st = -1,en = -1;
    std::vector<std::vector<std::pair<int, int>>> g; // 邻接表 (无向图)
    std::vector<pair<int,int>> fa; // [指向的父节点 fa, 父节点路径的权值 w] // 父节点记录
    // 1-base
    TreeDiameter(vector<vector<pair<int,int>>>& e) {
        n = e.size()-1;
        g = e;
        fa.resize(n+1);
    }
    // BFS 求最远节点 (返回节点编号)
    std::pair<int, int> bfs(int start)
    {
        std::vector<int> dep(n + 1, -1);
        std::queue<int> q;
        q.push(start);
        dep[start] = 0;
        fa[start] = {-1,0};
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (auto [v, w] : g[u])
            {
                if (dep[v] == -1)
                {
                    dep[v] = dep[u] + w;
                    fa[v] = {u,w};
                    q.push(v);
                }
            }
        }
        int maxPos = std::max_element(dep.begin(), dep.end()) - dep.begin();
        return std::make_pair(maxPos, dep[maxPos]);
    }
    // 求树的直径 (返回直径长度和端点 {u, v})
    std::pair<int, std::pair<int, int>> getDiameter()
    {
        if(d != -1){
            return {d,{en,st}};
        }
        auto [x, dis1] = bfs(1); // 第一次 BFS, 此时根变为 x
        auto [y, dis2] = bfs(x); // 第二次 BFS, 此时根变为 y
        d = dis2,st = y,en = x;
        return {dis2, {y, x}};
    }
    // 获取树直径上的所有点 first 以及 每个点到父节点路径的权值 second
    std::vector<pair<int,int>> getPathDia(){
        getDiameter();
        vector<pair<int,int>> path;
```

```

    int p = st;
    while(p != en){
        path.emplace_back(fa[p]);
        p = fa[p].first;
    }
    path.emplace_back(-1,0);
    return path;
}
};

```

直径的性质:

- 树上任意一点，离它最远的一点必然是直径两 endpoint 之一
- 两个联通块合并，新联通块的直径两 endpoint 一定是原先的四个直径 endpoint 中的其中两个
- 我们可以扩展 2 中的结论，我们将直径的定义扩展为点权 + 边权的情况下仍然成立
- 一棵树可以拥有多条直径，其所有直径共用一个中点，中点有可能在边上（两个相邻端点的边上）

在树上选择一条边长度和不超过  $s$  的路径使其他所有结点到这条路径的距离的最大值最小，那么选择的这条路径在树的直径上，可以用滑动窗口 + st 表做，窗口的两个 endpoint 处只需要维护直径上前后缀的长度即可

## 1.4 最近公共祖先 lca

### 1.4.1 重链剖分

重链剖分可以将树上的任意一条路径划分成不超过  $O(\log n)$  条连续的链，每条链上的点深度互不相同。同时会给树的点重新编号为  $dfn$  序。

$dfn$  序的定义：按先遍历重儿子的顺序得到的  $dfs$  序。

**重链剖分的性质：**

- 1: 每条重链上的  $dfn$  序连续。
- 2: 每个点子树内的  $dfn$  序连续，节点  $u$  子树内的  $dfn$  序范围是  $[dfn[u], dfn[u] + siz[u] - 1]$
- 3: 每个点到根节点的路径上，最多只会有  $O(\log n)$  个轻儿子。
- 4: 每条路径能被划分为最多  $O(\log n)$  条重链。

关于第三个和第四个性质的证明：

每次经过轻儿子，子树大小至少会减半，因此最多下降  $O(\log n)$  次。

```
template <class T>
class TreePre
{
private:
    int n;
    int idx = 0;
    int root;
    std::vector<std::vector<std::pair<int, int>>>> &g;
    void dfs1(int u, int f)
    {
        fa[u] = f;
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v != f)
            {
                dep[v] = dep[u] + 1;
                dis[v] = dis[u] + w;
                dfs1(v, u);
                siz[u] += siz[v];
                if (siz[son[u]] < siz[v])
                {
                    son[u] = v;
                }
            }
        }
    }
    void dfs2(int u, int tp)
    {
        dfn[u] = ++idx;
        idfn[idx] = u;
        top[u] = tp;
        if (son[u])
        {
            dfs2(son[u], tp);
        }
        for (auto [v, w] : g[u])
        {
            if (v != fa[u] and v != son[u])
```

```

        {
            dfs2(v, v);
        }
    }
}

public:
TreePre(std::vector<std::vector<std::pair<int, int>>> &g, int root)
    : g(g), n(g.size() - 1), root(root), dep(n + 1), top(n + 1), son(n + 1), fa(n + 1), siz(n + 1), dfn(0), idfn(0) {}
{
    dep[root] = 1;
    dis[root] = 0;
    dfs1(root, 0);
    dfs2(root, root);
}

std::vector<int> dfn, idfn, siz, fa, dep, top, son;
std::vector<T> dis;
int kthAncestor(int u, int k) // k 级祖先
{
    if (dep[u] <= k)
    {
        return -1;
    }
    int d = dep[u] - k;
    while (dep[top[u]] > d)
    {
        u = fa[top[u]];
    }
    return idfn[dfn[u] - dep[u] + d];
}

int getLca(int u, int v) // u 和 v 在指定根下的 lca
{
    while (top[u] != top[v])
    {
        if (dep[top[u]] > dep[top[v]])
        {
            u = fa[top[u]];
        }
        else
        {
            v = fa[top[v]];
        }
    }
    return dep[u] < dep[v] ? u : v;
}

T getDistance(int u, int v) // u 和 v 之间的距离
{
    return dis[u] + dis[v] - 2 * dis[getLca(u, v)];
}

bool isAncestor(int u, int v) // u 是不是 v 的祖先
{
    return dfn[u] <= dfn[v] and dfn[u] + siz[u] - 1 >= dfn[v];
}

auto getRoad(int u, int v) // 得到 uv 之间路径的 dfn 序
{

```

```

std::vector<std::pair<int, int>> res;
while (top[u] != top[v])
{
    if (dep[top[u]] < dep[top[v]])
    {
        std::swap(u, v);
    }
    res.push_back({dfn[top[u]], dfn[u]});
    u = fa[top[u]];
}
if (dep[u] > dep[v])
{
    std::swap(u, v);
}
res.push_back({dfn[u], dfn[v]});
return res;
}

int rootedFather(int u, int root) // u 在 root 为根的情况下的父节点
{
    if (!isAncestor(u, root)) // u 不是 root 的祖先 那么父节点不变
    {
        return fa[u];
    }
    auto it = std::upper_bound(g[u].begin(), g[u].end(), root, [&](int x, int y)
        { return dfn[x] < dfn[y]; });
    --it;
    return *it;
    // 把 u 的邻点按 dfn 序排序后 dfn 最大且 dfn[u] <= dfn[root] 的点
}

int rootedSize(int u, int root) // root 为根的情况下 u 的子树大小
{
    if (root == u)
    {
        return n;
    }
    if (!isAncestor(u, root)) // u 不是根的祖先
    {
        return siz[u];
    }
    return n - siz[rootedFather(u, root)];
}

int rootedLca(int root, int u, int v)
{
    return getLca(u, v) ^ getLca(u, root) ^ getLca(v, root);
}
};

```



### 1.4.2 树链剖分 (线段树)

配合区间修改线段树修改树上点权，如果修改边权可以让每个点的点权表示这个点指向父节点的边的权，那么让  $x$  到  $y$  路径的边权区间加可以先  $\text{addRoad}(x,y,v)$ ，再  $\text{addRoad}(\text{lca},\text{lca},-v)$ ;

```
template <class T>
class TreePre
{
private:
    int n;
    int idx = 0;
    int root;
    std::vector<std::vector<std::pair<int, int>>> &g;
    void dfs1(int u, int f)
    {
        fa[u] = f;
        siz[u] = 1;
        for (auto [v, w] : g[u]){
            if (v != f)
            {
                dep[v] = dep[u] + 1;
                dis[v] = dis[u] + w;
                dfs1(v, u);
                siz[u] += siz[v];
                if (siz[son[u]] < siz[v])
                {
                    son[u] = v;
                }
            }
        }
    }
    void dfs2(int u, int tp)
    {
        dfn[u] = ++idx;
        idfn[idx] = u;
        top[u] = tp;
        if (son[u]){
            dfs2(son[u], tp);
        }
        for (auto [v, w] : g[u]){
            if (v != fa[u] and v != son[u])
            {
                dfs2(v, v);
            }
        }
    }
public:
    TreePre(std::vector<std::vector<std::pair<int, int>>> &g, int root)
        : g(g), n(g.size() - 1), root(root), dep(n + 1), top(n + 1), son(n + 1), fa(n + 1), siz(n + 1), dfn(n + 1), idfn(n + 1)
    {
        dep[root] = 1;
        dis[root] = 0;
        dfs1(root, 0);
        dfs2(root, root);
    }
    std::vector<int> dfn, idfn, siz, fa, dep, top, son;
    std::vector<T> dis;
```

```

int getLca(int u, int v) // u 和 v 在指定根下的 lca
{
    while (top[u] != top[v]){
        if (dep[top[u]] > dep[top[v]]){
            u = fa[top[u]];
        }
        else{
            v = fa[top[v]];
        }
    }
    return dep[u] < dep[v] ? u : v;
}
// 对树上的路径操作
void addRoad(int x,int y,int v,SegmentTree<T>& tr){
    while(top[x] != top[y]){
        if (dep[top[x]] > dep[top[y]]){
            tr.rangeAdd( dfn[top[x]],dfn[x], v);
            x = fa[top[x]];
        }
        else{
            tr.rangeAdd( dfn[top[y]], dfn[y],v);
            y = fa[top[y]];
        }
    }
    if(dep[x] > dep[y]) swap(x,y);
    tr.rangeAdd(dfn[x], dfn[y], v);
}
T queryRoad(int x,int y,SegmentTree<T>& tr){
    int res = 0;
    while(top[x] != top[y]){
        if (dep[top[x]] > dep[top[y]]){
            res += tr.rangeQuerySum( dfn[top[x]],dfn[x]);
            res %= mod;
            x = fa[top[x]];
        }
        else{
            res += tr.rangeQuerySum( dfn[top[y]], dfn[y]);
            res %= mod;
            y = fa[top[y]];
        }
    }
    if(dep[x] > dep[y]) swap(x,y);
    res += tr.rangeQuerySum(dfn[x], dfn[y]);
    res %= mod;
    return res;
}
// [dfn[x], dfn[x]+siz[x]-1] 为 x 子树的所有节点
void addSubtree(int x,int z,SegmentTree<T>& tr){
    tr.rangeAdd(dfn[x], dfn[x]+siz[x]-1, z);
}
T querySubtree(int x,SegmentTree<T>& tr){
    int res = tr.rangeQuerySum(dfn[x], dfn[x]+siz[x]-1);
    return res;
}
};

```

### 1.4.3 欧拉序 + RMQ

我们考虑用  $dfs$  序求两个点的  $LCA$

结论：不妨设  $dfn[x] < dfn[y]$ ，那么  $lca(x, y)$  是  $dfn$  序上  $[dfn[x] + 1, dfn[y]]$  区间内深度最小的点的父亲

正确性显然：按  $x$  和  $y$  是否有祖先关系讨论即可。

用返回最小位置的  $ST$  表预处理可以做到  $O(1)$  查询。配合  $ST$  表板子食用（ $ST$  板子记得改成返回权值最小的下标）

重链剖分的预处理  $TreePre$  只需要  $dfs1, dfs2$  和构造函数就够了

```
template <class T>
class FastLCA
{
    int n;
    std::vector<std::vector<std::pair<int, int>>> g;
    TreePre<T> pre;
    ST<T> *st;

public:
    FastLCA(std::vector<std::vector<std::pair<int, int>>> &g, int root) : n(g.size() - 1), g(g), pre(g, root)
    {
        std::vector<int> reLabel(n + 1);
        for (int i = 1; i <= n; ++i)
        {
            reLabel[pre.dfn[i]] = pre.dep[i];
        }
        st = new ST(reLabel);
    }
    int getLca(int u, int v)
    {
        if (u == v)
        {
            return u;
        }
        u = pre.dfn[u], v = pre.dfn[v];
        if (u > v)
        {
            std::swap(u, v);
        }
        return pre.fa[pre.idfn[(*st)(u + 1, v)]];
    }
};
```

#### 1.4.4 倍增求 LCA

$O(\log(n))$  求 lca , 原理是先把  $u$  和  $v$  跳到同一个高度, 然后再共同上跳知道最近的 lca

```
void faLCA(int n, int root, vector<vector<int>>& adj, vector<vector<int>>& fa, vector<int>& dep) {
    fa = vector<vector<int>>(n + 1, vector<int>(21));
    dep = vector<int>(n + 1);
    auto dfs = [&](auto self, int u, int ffa) -> void {
        fa[u][0] = ffa;
        dep[u] = dep[ffa] + 1;
        for (int v : adj[u]) {
            if (v == ffa) continue;
            self(self, v, u);
        }
    };
    dfs(dfs, root, root);
    for (int k = 1; k <= 20; k++) {
        for (int i = 1; i <= n; i++) { fa[i][k] = fa[fa[i][k - 1]][k - 1]; }
    }
}

void solve() {
    int n, q, root;
    cin >> n >> q >> root;
    vector<vector<int>> adj(n + 1);
    for (int i = 2; i <= n; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].emplace_back(v);
        adj[v].emplace_back(u);
    }
    // static int fa[500010][21]; 洛谷模板题 P3379 换成这个才不会 t
    vector<vector<int>> fa;
    vector<int> dep;
    faLCA(n, root, adj, fa, dep);
    auto lca = [&](int u, int v) -> int {
        if (dep[u] < dep[v]) swap(u, v);
        for (int k = 20; k >= 0; k--) {
            if (dep[fa[u][k]] < dep[v]) continue;
            u = fa[u][k];
        }
        if (u == v) return u;
        for (int k = 20; k >= 0; k--) {
            if (fa[u][k] != fa[v][k]) { u = fa[u][k], v = fa[v][k]; }
        }
        return fa[u][0];
    };
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << lca(u, v) << "\n";
    }
}
```

#### 1.4.5 树上差分

**点权差分** ( $ans[x]$  表示  $x$  点的点权)

```
ans[x]++;ans[y]++;  
ans[LCA.pre.fa[lca]]-=1;  
ans[lca]-=1;
```

**边权差分** ( $ans[x]$  表示  $x$  到其父节点的边权)

```
ans[x]++;ans[y]++;  
ans[lca]-=2;
```

## 1.5 树上启发式合并

这是一个离线做法，本质就是对于每个节点都通过 *DFS* 求答案，但是通过保留重儿子的答案使复杂度降为优秀的  $n\log n$

具体做法是，对于以  $u$  为根的子树，我们先递归其轻儿子，但是信息我们不做保留，注意这次 *dfs* 并不是在求轻儿子的对  $u$  结点的答案，而是递归轻儿子求其自己的答案，递归结束后回到当前节点，我们再递归其重儿子，标记重儿子并且保留信息，然后我们开始统计该节点  $u$  的答案，由于重儿子的信息被保留，我们只需要计算轻儿子的贡献即可，这里实际上加上了轻儿子的贡献，统计完后在此更新该节点的答案，如果该节点  $u$  不是其父节点的重儿子，还需要清除贡献，清除贡献前置重儿子为空，这样会把该节点轻重儿子的贡献全部删去，保证了正确性

```
// 对于以  $u$  为根的子树
// 1 先统计其轻儿子的答案 统计完后删除信息
// 2 再统计其重儿子的答案 统计完后保留信息
// 3 将重儿子的信息合并到  $u$  上
// 4 再遍历  $u$  的轻子树 把轻子树的信息合并到  $u$  上
// 5 判断  $u$  的信息是否需要传给父节点（如果  $u$  是其父节点的重儿子就需要传递信息）

// 以求子树颜色种类为例
template <class T>
struct DSUOnTree
{
    int n;
    std::vector<std::vector<std::pair<int, int>>> g; // 邻接表
    std::vector<int> siz, son;
    std::vector<T> ans; // 子树大小、重儿子，每个节点的答案
    int root;
    int heavyNode = 0; // 当前处理的重儿子节点
    int typeCnt = 0; // 颜色种类数量
    std::vector<int> col, colorCnt; // 原始颜色数组 颜色出现次数数组
    DSUOnTree(std::vector<std::vector<std::pair<int, int>>> &g, std::vector<int> &col, int root)
        : n(g.size() - 1), g(g), siz(n + 1), son(n + 1), ans(n + 1),
          root(root), col(col), colorCnt(n + 1, 0) {}
    // 预处理重儿子
    void preProcess(int u, int f)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u]){
            if (v != f)
            {
                preProcess(v, u);
                siz[u] += siz[v];
                if (siz[son[u]] < siz[v])
                {
                    son[u] = v;
                }
            }
        }
    }
    // 添加/删除节点信息（需根据题目实现）
    // val 传 1 时是添加，传 -1 是删除
    void update(int u, int f, int val){
        // 示例：统计子树颜色种类数
        if (val == 1)
        {
            colorCnt[col[u]]++;
            typeCnt += (colorCnt[col[u]] == 1);
        }
        else
        {

```

```

        colorCnt[col[u]]--;
        typeCnt -= (colorCnt[col[u]] == 0);
    }
}

// 遍历子树统计/清除信息
void traverse(int u, int f, int val){
    update(u, f, val);
    for (auto [v, w] : g[u])
    {
        if (v != f && v != heavyNode)
        {
            traverse(v, u, val);
        }
    }
}

// 主算法
void solve(int u, int f, bool keep){
    // 1. 先处理轻儿子 (不保留信息)
    for (auto [v, w] : g[u]){
        if (v != f && v != son[u])
        {
            solve(v, u, false);
        }
    }
    // 2. 处理重儿子 (保留信息)
    if (son[u]){
        solve(son[u], u, true);
        heavyNode = son[u];
    }
    // 3. 再次遍历轻儿子并合并信息
    for (auto [v, w] : g[u]){
        if (v != f && v != son[u])
        {
            traverse(v, u, 1);
        }
    }
    update(u, f, 1); // 添加当前节点信息
    // 4. 此处可保存当前节点的答案
    ans[u] = typeCnt;
    heavyNode = 0;
    // 5. 如果需要清除信息
    if (!keep)
    {
        traverse(u, f, -1);
        typeCnt = 0;
    }
}

// 对外接口
void compute()
{
    preProcess(root, 0); // 假设根节点是 1
    solve(root, 0, true);
}

};

```

## 1.6 虚树

虚树是指我们在原本树的基础上抽离出一颗新的树，虚树只由关键点以及关键点的  $LCA$  构成，在预处理一些信息以后我们可以在虚树上求解只涉及关键点的答案。复杂度由  $O(n)$  降为  $O(V)$ ，其中  $V$  为关键点的数量。

构建方法：

因为多个点的  $LCA$  可能是同一个点，为了保证复杂度我们不能多次加入。

我们首先将关键点集合按  $dfn$  序排序，然后将相邻点的  $lca$  加入点集。

之后再次按  $dfn$  序排序并去重。然后把相邻的两个点  $(u, v)$  连边  $lca(u, v) -> v$

```
template <class T>
struct VirtualTreePre
{
    TreePre<T> pre;
    int root;
    std::vector<std::vector<std::pair<int, int>>> vt;
    VirtualTreePre(std::vector<std::vector<std::pair<int, int>>> &g, int root) : pre(g, root), root(root),
    void build(std::vector<int> a)
    {
        a.push_back(root); //加入根节点方便 DP, 根据题目情况判断是否加 root
        std::sort(begin(a), end(a), [&](int u, int v)
            { return pre.dfn[u] < pre.dfn[v]; });
        for (int i = a.size() - 1; i; --i)
        {
            a.push_back(pre.getLca(a[i], a[i - 1]));
        }
        std::sort(begin(a), end(a), [&](int u, int v)
            { return pre.dfn[u] < pre.dfn[v]; });
        a.erase(std::unique(begin(a), end(a)), end(a));
        for (auto u : a)
        {
            vt[u].clear();
        }
        for (int i = 1; i < a.size(); ++i)
        {
            int lca = pre.getLca(a[i], a[i - 1]);
            vt[lca].emplace_back(a[i], pre.dis[a[i]] - pre.dis[lca]);
        }
    }
};
```



## 1.7 树哈希

树哈希主要解决树的同构问题，即快速判断两颗树的结构是否相同

首先我们得有一个哈希函数  $f$ ，树哈希想要不被卡关键在于哈希函数的设置不能是线性函数  $ax + b$ ，我们可以设置成  $ax^3 + b$ ，对于整棵树哈希值的计算，我们的公式是

$$\text{hash}[u] = 1 + \sum_{v \in \text{son}(u)} f(\text{hash}[v])$$

有了这两个前置知识，我们分有根树和无根树来讨论

- 有根树，这种情况十分简单，我们从指定的根开始  $dfs$ ，求出根节点的哈希值，如果两个根节点的哈希值相同，那么两颗有根树同构
- 无根树，对于无根树的同构，我们有两种方法。

其一是先求出树的一个或两个重心，以重心为根  $dfs$  求出哈希值，如果有两个重心就把两个哈希值异或起来，然后对比即可，需要注意的是，如果存在两个重心，就必须求两个重心为根的哈希值，否则会出错 其二是求出以每个点为根的哈希值，然后异或起来得到最终哈希值，操作方法是朴素的换根  $DP$ ，模板代码给出的就是换根  $DP$  做法

```
using u64 = unsigned long long;
```

```
template <class T>
struct Rand
{
    std::mt19937 myrand;
    Rand(const long long seed = time(0)) : myrand(seed) {}
    T operator()(T l, T r)
    {
        return std::uniform_int_distribution<T>(l, r)(myrand);
    }
};

Rand<u64> rd;

u64 f(u64 x)
{
    const static u64 r1 = rd(1 << 20, 1 << 24);
    const static u64 r2 = rd(1 << 25, 1 << 30);
    const static u64 mask = (1ll << 31) - 1;

    auto h = [&](u64 y)
    {
        return (u64)y * y * y * r1 + r2;
    };
    return h(x & mask) + h(x >> 31);
}
```

```
struct Treehash // 得到每个点为根的 hash 值 On
{
    int n;
    std::vector<std::vector<int>>> g;
    std::vector<u64> hash;
    u64 ans = 0;
    void dfs1(int u, int fa)
    {
        hash[u] = 1;
        for (auto v : g[u])
        {
            if (v == fa)
            {
                continue;
            }
        }
    }
};
```

```

        }
        dfs1(v, u);
        hash[u] += f(hash[v]);
    }
}
void dfs2(int u, int fa)
{
    for (auto v : g[u])
    {
        if (v == fa)
        {
            continue;
        }
        hash[v] = hash[v] + f(hash[u] - f(hash[v]));
        dfs2(v, u);
    }
}
Treehash(std::vector<std::vector<int>> &e) : n(e.size() - 1), g(e), hash(n + 1)
{
    dfs1(1, 0);
    dfs2(1, 0);
    for (int i = 1; i <= n; ++i)
    {
        ans ^= hash[i];
    }
}
};

```

## 1.8 网络流

网络流是一种建好图以后，内置反悔机制的一种贪心策略，不需要我们人为去寻找贪心的反悔策略

容量: 当前边最多经过的流量记作  $C(u,v)$

流量: 当前边实际经过的流量记作  $F(u,v)$

残量:  $C(u,v)-F(u,v)$

残量网络: 把残量为 0 的边删去，剩下的图称作残量网络，对应边的边权也变为残量

增广路: 与二分图中增广路定义不同，增广路指残量网络上源点到汇点的一条简单路径

割: 割是一个边的集合，一个边集是割当且仅当去掉这些边后网络不在流通，即最大流为 0

最大流最小割定理: 一个网格的最大流等于其最小割

我们根据定义发现，只要存在增广路，流量就能不断增大，因此求最大流是一个求增广路的过程，并且这条增广路我们遵循能流满就流满的贪心策略

我们选择一条增广路，假设流量为  $f$ ，我们正向边容量  $-f$ ，反向边容量  $+f$ ，这就是最开始我们说的内置反悔机制，也就是说我们可以通过反边流量进行反悔

由于 EK 算法不优于 dinic，我们只介绍 dinic 算法

dinic 是多路增广 + 当前弧优化的增广路算法

我们通过 bfs 给图分层，分层后多路增广，也就是同时累加多条增广路的流量

当前弧优化是指，流量已经流满的边无用，我们没必要对每个点每次都从头开始遍历，而是记录一个当前弧，直接从当前弧开始遍历

贪心的正确性全部基于退流反边对流量不断调整直至最优状态

### 1.8.1 拆点

注意到在流网络中，我们对于限制都是加在边上的。那么对于点的限制我们往往考虑拆点。具体的说，我们把一个点  $U$  拆成对应的  $In_u$  和  $Out_u$ ，然后连边  $In_u \rightarrow Out_u$ ，边的流量限制即为该点的流量限制。

### 1.8.2 最大流 Dinic

```
template<class T>
struct MaxFlow{
    struct _Edge{
        int to;
        T cap;
        _Edge(int _to,T _cap):to(_to),cap(_cap){};
    };

    int n;
    std::vector<_Edge> e;
    std::vector<std::vector<int>> h;
    std::vector<int> cur,d;

    MaxFlow(){};
    MaxFlow(int _n){
        init(_n);
    }
    void init(int _n){
        n=_n;
        e.clear();
        h.assign(n,{});
        cur.resize(n);
        d.resize(n);
    }
};
```

```

void addEdge(int u,int v,T c){
    h[u].push_back(e.size());
    e.emplace_back(v,c);
    h[v].push_back(e.size());
    e.emplace_back(u,0);
}

bool bfs(int s,int t){
    d.assign(n,0);
    std::queue<int> q;
    d[s]=1;q.push(s);
    while(!q.empty()){
        int u=q.front();q.pop();
        for(int i:h[u]){
            auto [v,c]=e[i];
            if(c>0&&d[v]==0){
                d[v]=d[u]+1;
                if(v==t) return true;
                q.push(v);
            }
        }
    }
    return false;
}

T dfs(int u,int t,T f){
    if(u==t) return f;
    T sum=f;
    for(int &i=cur[u];i<(int)h[u].size();i++){
        int j=h[u][i];
        auto [v,c]=e[j];
        if(c>0&&d[v]==d[u]+1){
            T a = dfs(v,t,std::min(sum,c));
            e[j].cap-=a;
            e[j^1].cap+=a;
            sum-=a;
            if(sum==0) return f;
        }
    }
    if(sum==f) d[u]=0;
    return f-sum;
}

T Dinic(int s,int t){
    T ans=0;
    while(bfs(s,t)){
        cur.assign(n,0);
        ans+=dfs(s,t,std::numeric_limits<T>::max());
    }
    return ans;
}

std::vector<bool> minCut(){
    std::vector<bool> c(n);
    for(int i=1;i<n;i++){
        c[i]=(d[i]!=0);
    }
}

```

```

    }
    return c;
}

struct Edge {
    int from,to;
    T cap,flow;
};

std::vector<Edge> edges() {
    std::vector<Edge> a;
    for (int i = 0; i < e.size(); i += 2) {
        Edge x;
        x.from = e[i + 1].to;
        x.to = e[i].to;
        x.cap = e[i].cap + e[i + 1].cap;
        x.flow = e[i + 1].cap;
        a.push_back(x);
    }
    return a;
}
};

```

### 1.8.3 最小费用最大流

```
template<class T>
struct MinCostFlow {
    int n;

    struct Edge {
        int to;
        T cap, cost;

        Edge(int to, T cap, T cost) : to(to), cap(cap), cost(cost) {}
    };

    std::vector<Edge> e;
    std::vector<std::vector<int>>> h;
    std::vector<T> c, dis;
    std::vector<int> pre;

    MinCostFlow() {};

    MinCostFlow(int _n) {
        init(_n);
    }

    void init(int _n) {
        n = _n;
        e.clear();
        h.assign(n, {});
    }

    void addEdge(int u, int v, T cap, T cost) {
        h[u].push_back(e.size());
        e.emplace_back(v, cap, cost);
        h[v].push_back(e.size());
        e.emplace_back(u, 0, -cost);
    }

    bool dijkstra(int s, int t) {
        dis.assign(n, std::numeric_limits<T>::max());
        pre.assign(n, -1);
        std::priority_queue<std::pair<T, int>, std::vector<std::pair<T, int>>, std::greater<std::pair<T,
        dis[s] = 0, pq.emplace(0, s);
        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();
            if (dis[u] != d) {
                continue;
            }
            for (int i: h[u]) {
                auto [v, cap, cost] = e[i];
                if (cap > 0 && dis[v] > d + c[u] - c[v] + cost) {
                    dis[v] = d + c[u] - c[v] + cost;
                    pre[v] = i;
                    pq.emplace(dis[v], v);
                }
            }
        }
    }
};
```

```

        return dis[t] != std::numeric_limits<T>::max();
    }

    std::pair<T, T> flow(int s, int t) {
        T flow = 0, cost = 0;
        c.assign(n, 0);
        while (dijkstra(s, t)) {
            for (int i = 0; i < n; i++) {
                c[i] += dis[i];
            }
            T aug = std::numeric_limits<int>::max();
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                aug = std::min(aug, e[pre[i]].cap);
            }
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                e[pre[i]].cap -= aug;
                e[pre[i] ^ 1].cap += aug;
            }
            flow += aug;
            cost += aug * c[t];
        }
        return std::make_pair(flow, cost);
    }

    struct _Edge {
        int from;
        int to;
        T cap;
        T cost;
        T flow;
    };

    std::vector<_Edge> edges() {
        std::vector<_Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            _Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.cost = e[i].cost;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }
};

```